The Relevance Rule Decides the Leaderboard: Seven PDF Chunkers, Identical Chunks, Three Rankings

Aryaman Srivastava
srivastavaaryaman555@gmail.com
https://github.com/CandyButcher27/DocStruct
[TODO: affiliation, co-authors]

Abstract

Chunking decides what a retrieval-augmented system can retrieve, and chunkers are compared on leaderboards. We show those leaderboards are underdetermined by their own evaluation design. Scoring seven PDF chunkers on *identical chunks* over 95 documents and 3,558 human-authored questions, we find the ranking *inverts* depending only on the rule used to decide whether a retrieved chunk is relevant: the tool that places 1st of 7 under span and region relevance places 6th of 7 under page relevance, and the tool it displaces is its exact mirror. No relevance rule is size-neutral — span rewards large chunks, page rewards small ones — so a single-rule comparison reports a size preference and a ranking entangled, without separating them. We could find no prior chunking evaluation that varies the rule.

The instrument for this result is DocStruct, a fully local, deterministic, structure-aware chunker for born-digital PDFs. It fuses a rules-based geometric detector over PDF primitives with an optional DocLayNet-trained vision detector, recovers reading order and section hierarchy, and assembles chunks under a size floor. No language model is invoked on the parse path, and the resulting determinism is measured rather than asserted: across 95 documents parsed twice in independent processes, all 5,810 chunks per run were byte-identical.

To evaluate chunking without a relevance rule at all, we also score chunk boundaries against section boundaries authored by publishers in JATS XML for 134 PubMed Central papers — gold written for an unrelated purpose, before this benchmark existed — using the standard segmentation error rates Pk and WindowDiff. DocStruct leads both. We report where the approach does not pay: a vision detector that is significant on arXiv and null on three other corpora, $3.9\times$ the context cost of the leanest baseline, 81.7% word coverage against LangChain's 100%, and a widely used financial corpus that turns out to be unusable as a retrieval benchmark under a fixed-embedder protocol.

1 Introduction

A retrieval-augmented generation (RAG) system over PDFs is only as good as the units it retrieves. Those units — chunks — are produced by a pipeline that, in most deployments, is structurally blind twice over. A parser converts the PDF into a character stream, discarding the two-dimensional layout that told a human reader where a section began and which column to read next. A splitter then cuts that stream every N characters, with a separator preference list as the only concession to structure. Neither stage has a representation of "section", "table" or "column".

The consequences are visible in any real corpus. On a two-column paper, `page.extract_text()` welds the two columns into single unreadable lines; a chunk boundary lands mid-sentence because a character budget expired; a table is severed from the header row that gives its columns meaning. Each failure degrades retrieval in a way that is invisible to the parser's own metrics, because the parser was never asked to preserve structure — only characters.

Recent work responds by making segmentation semantic: an LLM judges where topics change [9], or an embedding model clusters sentences into coherent groups [26]. These methods improve on fixed-size splitting, but they buy that improvement with three costs that are rarely priced. They are non-deterministic, so the same PDF can yield different chunks on consecutive runs. They are expensive: one LLM or embedding call per candidate boundary, on every document, forever. And they are version-bound: the chunk boundaries are a property of a model checkpoint, so an upgrade silently invalidates an index. For a regulated or auditable pipeline — the setting where document structure matters most — these are disqualifying rather than inconvenient.

DocStruct takes the opposite position. Structure in a born-digital PDF is not a semantic inference problem; it is already present in the file, as font sizes, glyph positions, ruling lines and whitespace. Recovering it needs geometry and rules, not a language model. The contract is deliberate and total:

> *No LLM calls in the pipeline. The same PDF in, the same chunks out. Fully local, fully auditable.*

Contributions.

1. **Chunking leaderboards are underdetermined by their relevance rule, and we measure it.** On identical chunks over 95 documents and 3,558 human questions, the ranking of seven tools inverts between span, region and page relevance: DocStruct is 1st of 7 under two rules and 6th of 7 under the third, and unstructured is its exact mirror (§5.1). We can find no prior chunking evaluation that reports more than one rule. A sweep over the region threshold (§5.2) shows this is a property of the *rule*, not of a badly chosen constant.

(a) input page (b) fused blocks (2 seen by both detectors) (c) reading order + 4 chunks

Figure 1: DocStruct on one page of a two-column paper. (a) the page as the PDF ships it; (b) blocks after the geometric and vision detectors are fused, coloured by label; (c) the column-aware reading order and the chunks the blocks assemble into. Every box is an actual `run_pipeline()` result rather than an illustration, produced by `scripts/make_paper_figures.py`. Note that only 2 of 12 regions on this page were seen by both detectors — the two are genuinely blind to different things, which is what makes their agreement informative.

2. **A way to evaluate chunkers with no relevance rule at all.** We score chunk boundaries against section boundaries authored by publishers in JATS XML for 134 PubMed Central papers, using the standard segmentation error rates Pk and WindowDiff (§5.3). No embedder, no relevance rule, and gold written for an unrelated purpose before this benchmark existed.

3. **DocStruct, the instrument**: a deterministic, fully local, structure-aware chunker that fuses a geometric and a vision layout detector, recovers reading order and section hierarchy, and assembles chunks under a size floor (§3). Its determinism is measured, not asserted — 95 documents, two independent processes, 5,810 byte-identical chunks per run (§5.4) — which is a row no LLM-segmented or embedding-clustered chunker can fill.

4. **Results against our own system, reported in the main text** (§6): the vision detector is significant on arXiv and null on three other corpora; our context cost is $3.9\times$ the leanest baseline; our word coverage is 81.7% against LangChain's 100%; and a widely-used financial corpus turns out to be unusable as a retrieval benchmark under a fixed-embedder protocol, where a published paper reports MRR 0.700–0.844 on the same files.

2 Related Work

Parse fidelity benchmarks. OmniDocBench [20] and READoc [17] evaluate whether a converter reproduces a document's text, tables and reading order, using normalised edit distance, TEDS and vocabulary F1. Docling [3, 19] and the tool comparison of Adhikari and Agarwal [2] sit in the same family. These measure the parse; they stop before chunking, and therefore before retrieval. DocStruct is complementary: we do not currently report a parse-fidelity number, which we list as a limitation (§6.5).

Layout datasets and detectors. PubLayNet [29], DocBank [16] and DocLayNet [22] provide page-level layout supervision; PubTables-1M [27] and SciTSR [7] target tables. Our optional vision detector is a YOLO model trained on DocLayNet. We use such a detector as *one of two* evidence sources rather than the sole one, which is what allows the pipeline to degrade to a rules-only mode with no model weights present.

Chunking strategies. LumberChunker [9] uses an LLM to locate topic shifts. Chroma's technical report [26] introduces a cluster-based semantic chunker and, importantly for us, a token-level evaluation vocabulary (IoU, Precision$_\Omega$). Jimeno-Yepes et al. [15] show that element-type chunking beats fixed-size splitting on financial reports — the closest prior claim to ours. Śmigielski et al. [25] ask whether semantic chunking earns its compute and largely conclude that it does not, which supports our determinism argument. Bennani and Moslonka [5] report that overlap gives little benefit and identify a "context cliff" past roughly 2,500 tokens; Bhat et al. [6] find chunk size dominates other choices. A 2026 wave of structure-aware chunkers [1, 12, 18] shares our positioning and post-dates our design.

End-to-end parse $\times$ chunk $\times$ RAG. Closest to our evaluation setting, El Bachyr et al. [11] benchmark parser and chunker combinations for financial QA. OHR-Bench [28] measures how OCR noise propagates into RAG, and supplies the human-annotated corpus we adopt as our primary external benchmark. We differ from both in a specific way that §5.1 makes concrete: they report a single relevance rule, and we show that the choice of rule can reverse the ranking.

Segmentation metrics. Pk [4] and WindowDiff [21] are the standard error rates for linear text segmentation. To our knowledge they have not previously been used to evaluate RAG chunkers against publisher-authored document structure, which is what §5.3 does.

3 The DocStruct Pipeline

DocStruct is a six-stage pipeline. Every threshold it uses lives in a single `config.py`, and every tuned constant carries a comment naming the measurement that chose it.

3.1 Two mutually blind detectors

The *geometric* detector reads PDF primitives directly through `pdfplumber`: character positions, font names and sizes, ruling lines and rectangles. It proposes blocks from font-size clustering, vertical whitespace runs and ruling geometry. It is exact where the PDF is well-formed and blind where structure is implied only visually — a borderless table, for instance, has no ruling lines to find.

The *vision* detector is a YOLOv8m trained on DocLayNet, run over page rasters. It proposes the same block types from appearance alone. It is blind to everything the geometric detector reads directly, and it is optional: with no weights present the pipeline runs geometry-only, which we report throughout as DocStruct-geo.

The two are mutually blind by construction, which is what makes their agreement informative rather than redundant.

3.2 Fusion

Proposals are matched by IoU and merged into a single sequence of blocks, each carrying a confidence derived from whether one or both detectors saw it and how well they agreed. Coordinates are top-left throughout (y_0 = top, y increasing downward), matching `pdfplumber`; model output is transformed on the way in. The scale factors applied to a unilateral detection, and the clamp ranges on the resulting confidence, are inherited from a prototype and are still marked `# unvalidated` in `config.py`. They were never tuned against annotated layout gold, because we have none at the scale required; §6.5 returns to this.

3.3 Reading order and hierarchy

Blocks are ordered by a column-aware sweep that detects column bands from the x distribution and reads each band top-to-bottom before moving right. Headings are classified by relative font size and weight, and each block is assigned a `SectionPath` — the stack of headings above it. We evaluated recursive XY-cut as an alternative and *rejected* it: it cost -0.0101 MRR on our internal corpus (§5.5).

3.4 Extraction and chunk assembly

Text is extracted per block with a font-scaled word-gap tolerance, which recovers inter-word spacing that a fixed tolerance mis-splits on large display type.

Chunks are then assembled by walking blocks in reading order and closing a chunk when a structural boundary is reached *and* the accumulated content exceeds `MIN_CHUNK_TOKENS`. This floor is the single most valuable decision in the system: flushing on every structural boundary was worth 0.6890 MRR, and adding the floor together with inline headers moved it to 0.7319 ($+0.0429$). Tables are emitted as their own chunks with their header rows attached.

4 Experimental Setup

4.1 Protocol: hold everything but the chunker fixed

Every tool in every table below is scored through the same harness. The embedder (`all-MiniLM-L6-v2`, 23), the lexical retriever (BM25, 24), the fusion (RRF with k=60, 8) and k=5 are identical across tools. The chunker is the only variable. We report MRR, nDCG@5 [14], Recall@5 and Precision@1, each with a bootstrap 95% CI, and *paired* bootstrap p-values against DocStruct [10]. Paired tests matter here: comparing overlapping marginal CIs is the standard way to wrongly call a real difference insignificant.

4.2 Corpora

OHR-Bench [28] is our primary external benchmark: 95 born-digital documents across law, manuals, finance and academia, with 3,558 human-authored questions and human-annotated evidence. It is the headline because the gold is human, external, and predates our system.

PubMed Central supplies 134 open-access papers from seven journals, each fetched *with its publisher's JATS XML*. The XML carries the real section hierarchy, authored by the publisher and never touched by a chunker. This is the strongest form of tool-agnostic gold available to us: it is not merely not-ours, it predates the benchmark and was written for an unrelated purpose.

An internal corpus of 92 born-digital arXiv PDFs with 558 LLM-generated question–span pairs is retained for ablations only. Its gold is generated, so per our own rule it never carries a headline number.

FinanceBench [13] was intended as a second external leaderboard and is *not* used as one. §6.4 reports why.

4.3 Relevance modes

Deciding whether a retrieved chunk is "relevant" requires a rule, and the choice is not innocuous. We implement three:

- **span** — the chunk contains the gold answer span verbatim (whitespace-blind).
- **region** — the chunk overlaps the gold evidence region by at least `RELEVANCE_REGION_MIN_OVERLAP` $= 0.7$.
- **page** — the chunk comes from the page the gold is annotated on.

None is size-neutral. **span** rewards large chunks, since a longer chunk is more likely to contain any given span. **page** rewards small chunks, since a short chunk is more likely to sit

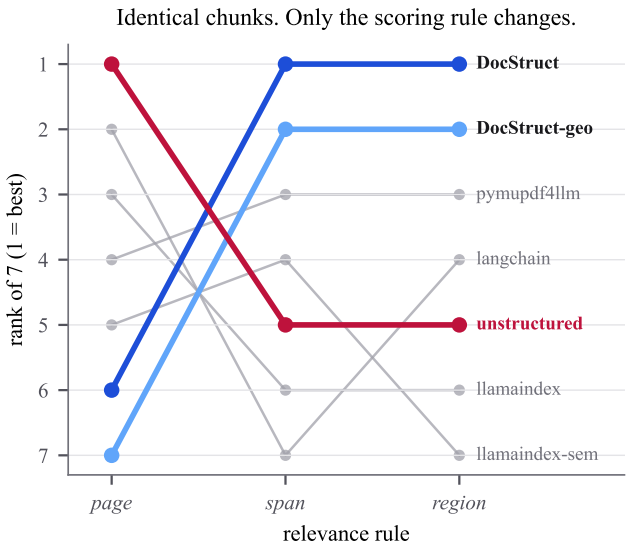

Figure 2: The same seven chunkers, the same chunks, three relevance rules. DocStruct runs 6th → 1st → 1st and unstructured runs 1st → 5th → 5th. Nothing about the chunks changed between these columns; only the rule that decides whether a retrieved chunk counts as relevant.

entirely on one page. **region** is the only one designed to be size-tolerant. §5.1 shows this is not a theoretical concern.

4.4 Reachability: a ceiling, not a score

Before any leaderboard, we measure what fraction of a corpus's gold is findable in the PDF's own text at all. This is identical for every tool, so it is a ceiling rather than a comparison, and it says whether a rule can measure the corpus. On OHR-Bench, 80.2% of gold is span-reachable. The check also carries a circularity warning: once gold occupies a large share of its page — 69% on FinanceBench — span and region reachability approach 100% by construction and prove nothing.

5 Results

5.1 The relevance rule decides the winner

Figure 2 and Table 1 are the paper's central result. The three runs share *identical chunks* — chunk counts and mean chunk widths are equal across modes, because chunking came from a warm cache and only scoring changed — so the relevance rule is the only variable.

The ranking inverts. DocStruct is 1st of 7 under span and region, and 6th of 7 under page. Unstructured is the exact mirror: 1st under page, 4th and 5th under the other two. This is not noise — under region, DocStruct beats all five external tools significantly; under span it beats four of five, with `pymupdf4llm` inside the noise ($p = 0.23$); under page it loses to four of five.

Table 1: OHR-Bench: 95 documents, 3,558 human questions, seven chunkers, MRR under three relevance rules on *identical chunks*. Rank in parentheses. The ranking inverts with the rule.

Tool	page	span	region	ctx
DocStruct	0.600 (6)	**0.706 (1)**	**0.666 (1)**	2194
DocStruct-geo	0.470 (7)	0.705 (2)	0.657 (2)	2328
pymupdf4llm	0.668 (4)	0.699 (3)	0.604 (3)	2424
unstructured	**0.795 (1)**	0.654 (5)	0.601 (5)	**561**
langchain	0.756 (2)	0.641 (7)	0.603 (4)	638
llamaindex	0.729 (3)	0.648 (6)	0.589 (6)	1430
llamaindex_sem	0.652 (5)	0.654 (4)	0.575 (7)	4698

Table 2: OHR-Bench region MRR against the relevance threshold. A DocStruct variant is 1st at all ten thresholds; the two hold both top places at all ten.

Tool	0.1	0.3	0.5	0.7	0.9	1.0
DocStruct	.959	.880	.789	**.666**	**.528**	**.389**
DocStruct-geo	**.967**	**.893**	**.800**	.657	.508	.370
pymupdf4llm	.946	.854	.747	.604	.482	.326
llamaindex_sem	.962	.866	.740	.575	.430	.310
unstructured	.937	.834	.721	.601	.391	.213
llamaindex	.946	.841	.726	.589	.460	.340
langchain	.936	.814	.711	.603	.466	.337

The mechanism is chunk size, and it is not subtle. Unstructured emits the smallest chunks in the field and wins the mode that rewards small chunks. DocStruct emits large structural units and wins the modes that reward them. Under page mode DocStruct-geo scores 0.470 against hybrid DocStruct's 0.600 purely because geometry-only emits 5,810 chunks to hybrid's 9,080 — a +0.13 "improvement" from the vision detector that is an artefact of chunk count, not of quality.

What follows for the field. A chunking leaderboard that reports one relevance rule has not reported a ranking; it has reported a ranking *and* a size preference, entangled. We could find no prior chunking evaluation that varies the rule. We therefore publish all three modes, and we recommend the corpus's gold shape choose the rule: span where gold marks a verbatim sentence, region where it marks a block.

5.2 The region threshold does not choose the winner

The obvious attack on the region column of Table 1 is that `RELEVANCE_REGION_MIN_OVERLAP` = 0.7 was an unvalidated constant. We tested it by dumping the continuous overlap behind every retrieved chunk in one run and re-scoring offline at every threshold, so chunking is identical at every point. The dumping run reproduces the published leaderboard to a maximum MRR drift of 0.0002 with identical chunk counts, which is the precondition for the sweep meaning anything.

A DocStruct variant places 1st at all ten thresholds, and the two hold both top places at all ten, with a margin of +0.045

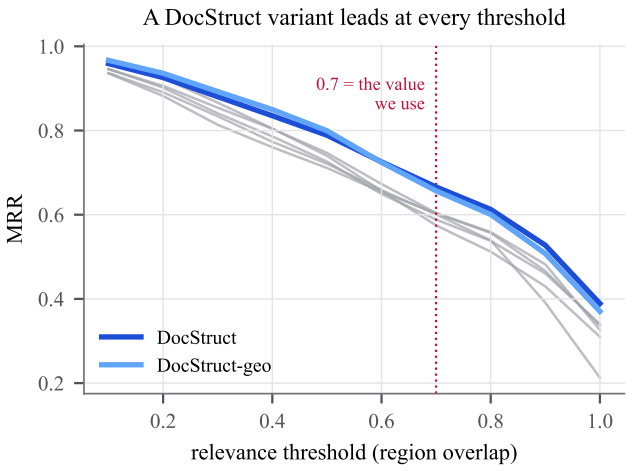

Figure 3: Region MRR against the relevance threshold, re-scored offline from one run's dumped overlaps so the chunks are identical at every point. The two DocStruct lines are on top across the whole range. Read the plateau, not the peak: every curve rises as the threshold falls because at 0 every chunk is relevant by definition.

to +0.062 over the best external tool across 0.4–1.0. The region result does not depend on the constant.

Two caveats belong here rather than in a rebuttal. First, 0.7 **is where our margin happens to peak** (+0.0619); the value predates the sweep by months, but the coincidence should be stated by us and not discovered by a reader. The defence is that the margin is +0.045 or better everywhere above 0.4, so the peak is a bump on a plateau. Second, **the low end is uninformative rather than favourable**: at threshold 0 every chunk is relevant and MRR is 1.0 by definition, so the convergence at 0.1 is that definition asserting itself. A metric that rises as a threshold falls is not evidence for a low threshold.

Finally, note that *the field below us reorders constantly* — `llamaindex_sem` is 2nd at 0.1 and 7th at 0.7; unstructured is 4th at 0.6 and 7th at 1.0. Our position is threshold-independent; a published ranking of the whole field is not. This is the lesson of §5.1 one level down.

5.3 Section-boundary agreement, with no retriever

Every result so far passes through an embedder and a relevance rule. This one does not. We compare each chunker's boundaries to the section boundaries the publisher wrote in JATS XML, using Pk [4] and WindowDiff [21]. Both are *error* rates: lower is better.

DocStruct-geo leads both metrics. It does so on a corpus where the gold was written by publishers for their own typesetting, which is the strongest tool-agnostic guarantee in this paper.

The result is stable under a 5.6× corpus increase. A 24-document pilot produced the identical ordering with every value within 0.02 (WindowDiff 0.436 → 0.423, Pk 0.353 → 0.342). We report this because a segmentation metric on a small corpus is exactly the kind of number that fails to replicate.

Table 3: Section-boundary agreement against publisher JATS gold, 134 PubMed Central papers. Pk and WindowDiff are error rates (lower is better). Straddle rate is descriptive, not an error. Unstructured's row covers 99 documents; it hard-failed on 35.

Tool	WinDiff	Pk	Strad.	Chunks
DocStruct-geo	**0.423**	**0.342**	0.513	26.8
pymupdf4llm	0.480	0.449	0.573	17.7
DocStruct	0.482	0.353	0.439	37.5
llamaindex_sem	0.534	0.513	0.189	29.1
llamaindex	0.695	0.598	0.366	42.7
langchain	0.879	0.620	0.220	85.6
unstructured	0.893	0.603	0.182	106.9

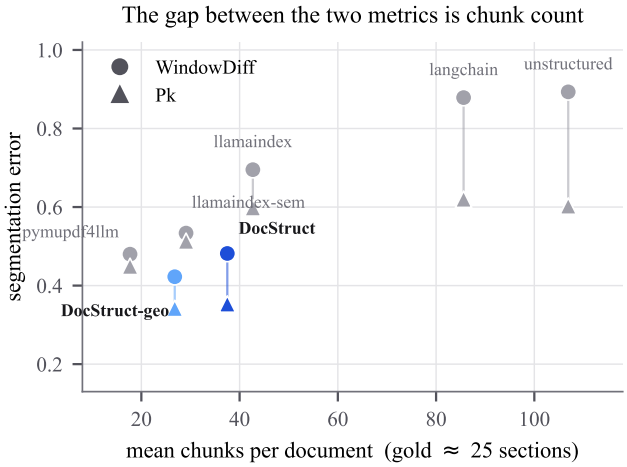

Figure 4: Why Pk and WindowDiff must be read together. Each tool contributes two marks joined by a line; the length of that line is how much the two metrics disagree, and it grows with chunk count. LangChain and unstructured emit 86 and 107 chunks against a gold averaging ≈25 sections, so WindowDiff — which counts boundaries per window — punishes them where Pk does not.

Read Pk and WindowDiff together or not at all. (Figure 4.) WindowDiff counts boundaries per window, so it punishes a tool that puts three splits where the document has one; Pk only asks whether a window's ends share a segment, so it forgives over-segmentation. LangChain (86 chunks) and unstructured (107) against a gold averaging ≈25 sections are the case in point: they are last under WindowDiff and mid-table under Pk. Either metric alone is quotable in the opposite direction.

Straddle rate is descriptive, not an error term. 57.4% of gold sections are shorter than `MIN_CHUNK_TOKENS`, so merging them is the design working as specified. The rate bounds how meaningful a per-chunk section *label* can be; it is not a score, and our 0.513 is not a win to claim.

Two honest qualifications. The ceiling is 84.7% of body sections locatable in the PDF's own text over 138 documents, of which 134 scored — scores must be read against that, not

against 100%. And unstructured's row covers 99 of 134 documents because it hard-failed on 35 (26%); the rate matched the pilot's $6/24$, so it is systematic. That failure rate is itself a finding about unstructured on born-digital PDFs, and we report it rather than silently dropping the tool.

5.4 Determinism, measured

Every claim above is about quality. This one is about the property that motivated the design, and it is the claim a reader should be most suspicious of, because determinism is easy to assert and rarely tested past a single file.

We parsed all 95 OHR-Bench documents twice, each parse in a *separate process*, and compared a hash over the full chunk structure: identifier, type, page, reading order, section path, and a digest of the content. Two runs agree only if every chunk matches on all of them. Separate processes matter: a same-process check cannot observe anything that varies across a process boundary, which is where non-determinism comes from — hash seeds, iteration order over addresses, thread scheduling, kernel selection.

All 95 documents were byte-identical, across 5,810 chunks per run — 100%, with zero differing documents. Three dense financial filings needed a raised per-parse cap to finish at all, and §6.5 reports that cost; none of them disagreed.

The chunk total is an unplanned cross-check. 5,810 is exactly the chunk count recorded for DOCSTRUCT-GEO in the OHR-Bench retrieval run of Table 1, produced days earlier on different hardware by a different code path. Two independent runs agreeing to the chunk is the kind of evidence a determinism claim wants.

Two limits belong with the claim rather than after it. Determinism holds *within* a version: a release that changes chunking changes boundaries, so a persisted index must pin a version. And this run is geometry-only. The hybrid path runs a model through CUDA, whose kernel selection is not guaranteed bit-reproducible, and we did not have a GPU available to test it — so the guarantee is established for DOCSTRUCT-GEO and remains unverified for DOCSTRUCT. Geometry-only is pure Python and NumPy and has no such caveat.

To our knowledge no LLM-segmented or embedding-clustered chunker can report this row at all, which is the practical content of the determinism argument: not that determinism is elegant, but that it is checkable, and we checked it.

5.5 Internal corpus and ablations

Table 4 reports the internal arXiv corpus. Its gold is LLM-generated, so it carries no headline claim; we use it for ablations, where only differences between our own configurations matter.

Where the gain came from. Starting from 0.6773, the chunk-boundary floor with inline headers gave $+0.0429$, and font-scaled word-gap tolerance a further $+0.0138$. Recursive XY-cut *cost* -0.0101 and is off by default. We report the rejected change because a table of only-successes is not an ablation.

Table 4: Internal corpus (92 arXiv PDFs, 558 generated Q&A). Used for ablations only — the gold is generated, so this is not a headline number.

Tool	MRR	nDCG	R@5	Cov.	Dup.
DOCSTRUCT	**0.820**	**0.832**	**0.943**	0.817	2.06
DOCSTRUCT-GEO	0.776	0.799	0.928	0.822	1.33
pymupdf4llm	0.765	0.790	0.919	0.768	1.36
langchain	0.701	0.728	0.848	**1.000**	**1.10**
unstructured	0.695	0.727	0.856	0.833	1.38

A measurement artefact that reversed a conclusion. An earlier corpus (55 documents) reported the vision detector at $+0.0092$ MRR, $p = 0.64$ — "the model does not pay for itself". That was wrong, and wrong for an instructive reason: `page.extract_text()` welded two-column pages into unquotable lines, so gold drawn from two-column pages — exactly where the vision detector helps most — was being silently rejected during gold generation. Fixing reference extraction moved the effect to $+0.0443$, $p = 0.0026$. The earlier numbers are superseded. We include this because it is a concrete case of a measurement defect flipping a conclusion, and because it motivates the reachability ceilings we now compute before every leaderboard.

6 Where the approach does not pay

6.1 The vision detector does not generalise

On the internal arXiv corpus, the vision detector is worth $+0.0443$ MRR ($p = 0.0026$), which is significant. It does not replicate. On OHR-Bench the same ablation gives $+0.0012$ under span ($p = 0.80$) and $+0.0090$ under region ($p = 0.12$), neither significant. On the PMC section metric, geometry-only *beats* the hybrid on WindowDiff at $2.4\times$ the speed. Three corpora now find nothing.

The honest statement is therefore: the vision detector is an arXiv result, not a property of the design. The one place it shows a sign of life is the strict half of the threshold sweep, where hybrid overtakes geometry-only between 0.5 and 0.6 ($+0.0092$ at 0.7) — small, not significant, but in the direction a table-heavy corpus would predict.

6.2 We are expensive in context

Figure 5 shows the trade. DOCSTRUCT retrieves 2,194 context words per query against unstructured's 561. On rank quality per token spent, unstructured is $3.9\times$ more efficient. We win accuracy and lose token cost, and for a cost-sensitive deployment that trade may not be worth making. This is a real axis on which we lose and it belongs in the main table, not an appendix.

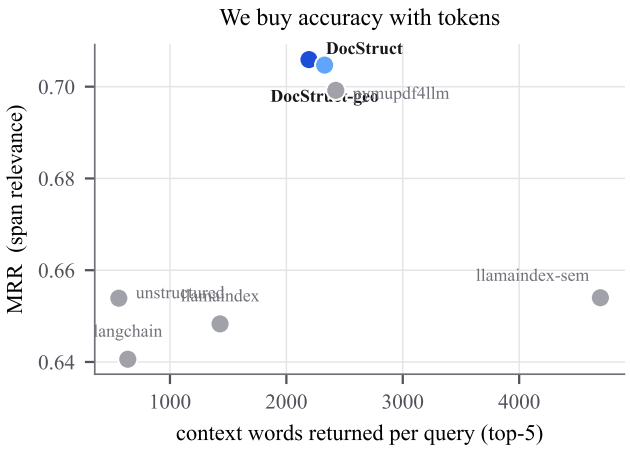

Figure 5: Accuracy against the token bill. DocStruct sits top-centre: best MRR, but roughly four times the context of unstructured and langchain, which sit bottom-left. A deployment that pays per token may reasonably prefer the bottom-left corner.

6.3 We discard document text

Word coverage is 0.817 against LangChain's 1.000: raw-text splitting drops nothing by construction, and we drop references, headers and footers by design. Our chunk duplication is 2.06×, the highest in the comparison, from inline headers and separately emitted table chunks. DocStruct optimises retrieval, not raw preservation.

6.4 FinanceBench is not a retrieval benchmark for us

We intended FinanceBench [13] as a second external leaderboard. It cannot serve as one under our protocol. Holding chunks fixed and varying only the embedder, the rank of the first relevant chunk was 81–299 (`all-MiniLM-L6-v2`), 63–181 (`bge-small-en-v1.5`) and 42–252 (`e5-small-v2`). Recall@5 was 0/5 under all three; MRR@5 was 0.0000 under all three.

The scoring rule is not at fault: a relevant chunk exists at region overlap 0.99–1.00. Retrieval simply never reaches it. FinanceBench questions are analyst prompts with almost no lexical overlap with their evidence, the evidence is a numeric table that small encoders embed poorly, and a 10-K yields 729–1,176 chunks. Every tool would score ≈ 0, which discriminates nothing — and a table of zeros reads as a chunking result when it is a retrieval limit.

We report this as a finding rather than omitting the corpus, because El Bachyr et al. [11] report MRR 0.700–0.844 on these same PDFs. Our protocol reaches 0.0 on the same files. That makes the usual "not comparable — different retrievers, page-level gold" caveat concrete rather than rhetorical, and it is a caution to anyone reading cross-paper numbers on this corpus.

6.5 Standing limitations

Born-digital only. DocStruct reads PDF primitives, so scanned documents without a text layer are out of scope. We do not run OCR.

No parse-fidelity number. We do not report edit distance or TEDS on OmniDocBench or READoc, so "the parse is correct" is currently asserted through retrieval quality rather than measured directly.

Detection is under-validated. Our layout metric rests on two hand-annotated documents. The DocLayNet val split is the correct home for it and is not yet run. Several fusion constants remain marked `# unvalidated`.

Academic is our weakest retrieval domain. On OHR-Bench span, DocStruct scores 0.453 against unstructured's 0.515 — 5th of 7 — on a 10-document slice too thin to settle. "DocStruct is good for research papers" is not what the retrieval data says, even though the PMC structural result is strong.

Performance, and it is our sharpest practical limitation. A figure-dense 18-page paper takes roughly 8 minutes to parse, dominated by materialising 422k vector primitives, and this is on the user-facing `parse()` path rather than only the benchmark. The determinism run (§5.4) put a number on the tail: three of 95 documents exceeded a 30-minute per-parse cap and needed four hours to finish, all dense financial filings — `JPMORGAN_2022Q2_10Q` (197 pages), `JPMORGAN_2023Q2_10Q` (217) and `VERIZON_2021_10K` (120) — against a corpus median of 18 pages. Page count alone does not explain it: the corpus's longest document, at 382 pages, parsed without trouble. Table density does. These are the same documents FinanceBench is built from, so the cost is measured on two independent corpora rather than inferred from one anecdote, and it is a real barrier to using DocStruct on long financial documents today.

Single-column English bias. IEEE two-column templates are unrepresented in our corpora, and all corpora are English.

7 Reproducibility

Code, configuration and every report cited above are public at `https://github.com/CandyButcher27/DocStruct`. All three corpora are fetched by scripts in the repository rather than redistributed, so the exact corpora can be reconstructed:

- `scripts/fetch_ohrbench.py` — OHR-Bench, 95 PDFs
- `scripts/fetch_pmc.py` — PMC papers with JATS XML
- `scripts/fetch_financebench.py` — FinanceBench filings

`datasets/` in the repository root carries the manifests, SHA-256 checksums and derived gold files needed to verify a reconstruction byte-for-byte. The two GPU

runs behind §5.2 and §5.3 are reproduced end-to-end by `notebooks/pmc_sections_colab.ipynb`. Determinism is checked by `scripts/verify_determinism.py`, which reproduces §5.4 end-to-end:

```
python scripts/verify_determinism.py \
--pdfs-dir data/ohrbench --runs 2
```

The library is installable as `pip install docstruct-rag`; the import name is `docstruct`.

8 Conclusion

The result we would most like carried forward is not a leaderboard position. It is that *the leaderboard depends on the relevance rule* — measurably, reversibly, on chunks that never changed. A chunking comparison that reports one rule has reported a ranking and a size preference entangled together, and since no rule is size-neutral, that entanglement is not a detail. It can reverse the conclusion. Until reporting more than one rule is standard practice, chunker comparisons — including ours — should be read with the rule in view.

Two smaller things follow from having built an instrument careful enough to show it. Publisher-authored JATS structure gives a way to score chunk boundaries with no retriever and no relevance rule in the loop, which sidesteps the question of who wrote the gold. And determinism, when a pipeline has no model on its parse path, is not a slogan but a property that can be checked in an afternoon: 95 documents, two independent processes, 5,810 identical chunks.

DOCSTRUCT itself is a competent structure-aware chunker that wins where the rules favour large structural units and loses where they favour small ones. That is a narrower claim than "it is the best chunker", and it is the one the evidence supports.

Open work. Two items on our pre-submission list remain undone, and both are measurement rather than writing. **Token-level IoU and Precision$_\Omega$** [26] are the published vocabulary for "rank quality per token spent" and would replace the homemade MRR-per-1k-words column in Figure 5; our dumped runs record the overlap *score* behind each retrieved chunk but not the chunk and gold token sets, so the metric needs a re-run rather than a re-analysis, and we would rather report it missing than approximate it. **DocLayNet's validation split** is the right home for the detection-layer mAP that currently rests on two hand-annotated documents, and the only cheap route to calibrating the fusion constants of §3.

References

[1] MultiDocFusion: Hierarchical and multimodal chunking pipeline for enhanced RAG on long industrial documents.

[2] Narayan S. Adhikari and Shradha Agarwal. A comparative study of PDF parsing tools across diverse document categories. *arXiv preprint arXiv:2410.09871*, 2024.

[3] Christoph Auer, Maksym Lysak, Ahmed Nassar, et al. Docling technical report. *arXiv preprint arXiv:2408.09869*, 2024.

[4] Doug Beeferman, Adam Berger, and John Lafferty. Statistical models for text segmentation. *Machine Learning*, 34(1–3):177–210, 1999. doi: 10.1023/A:1007506220214. URL `https://link.springer.com/article/10.1023/A:1007506220214`.

[5] Sofia Bennani and Charles Moslonka. A systematic analysis of chunking strategies for reliable question answering. *arXiv preprint arXiv:2601.14123*, 2026.

[6] Sinchana Ramakanth Bhat, Max Rudat, Jannis Spiekermann, and Nicolas Flores-Herr. Rethinking chunk size for long-document retrieval: A multi-dataset analysis. *arXiv preprint arXiv:2505.21700*, 2025.

[7] Zewen Chi, Heyan Huang, Heng-Da Xu, Houjin Yu, Wanxuan Yin, and Xian-Ling Mao. Complicated table structure recognition. *arXiv preprint arXiv:1908.04729*, 2019.

[8] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, 2009.

[9] André V. Duarte, João Marques, Miguel Grac ca, Miguel Freire, Lei Li, and Arlindo L. Oliveira. LumberChunker: Long-form narrative document segmentation. *arXiv preprint arXiv:2406.17526*, 2024.

[10] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[11] Omar El Bachyr, Yewei Song, Saad Ezzini, Jacques Klein, Tegawendé F. Bissyandé, Anas Zilali, Ulrick Ble, and Anne Goujon. Empirical evaluation of PDF parsing and chunking for financial question answering with RAG. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026. doi: 10.1145/3786583.3786911. arXiv:2604.12047.

[12] Pooja Guttal, Varun Magotra, Vasudeva Mahavishnu, Natasha Chanto, Sidharth Sivaprasad, and Manas Gaur. Structure-aware chunking for tabular data in retrieval-augmented generation. *arXiv preprint arXiv:2605.00318*, 2026.

[13] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

[14] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. In *ACM Transactions on Information Systems*, 2002.

[15] Antonio Jimeno-Yepes, Yao You, Jan Milczek, Sebastian Laverde, and Renyu Li. Financial report chunking for effective retrieval augmented generation. *arXiv preprint arXiv:2402.05131*, 2024.

[16] Minghao Li, Yiheng Xu, Lei Cui, Shaohan Huang, Furu Wei, Zhoujun Li, and Ming Zhou. DocBank: A benchmark dataset for document layout analysis. In *International Conference on Computational Linguistics (COLING)*, 2020. arXiv:2006.01038.

[17] Zichao Li et al. READoc: A unified benchmark for realistic document structured extraction. In *Findings of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2409.05137.

[18] Xiaoyu Liu. TopoChunker: Topology-aware agentic document chunking framework. *arXiv preprint arXiv:2603.18409*, 2026.

[19] Nikolaos Livathinos, Christoph Auer, Maksym Lysak, et al. Docling: An efficient open-source toolkit for AI-driven document conversion. *arXiv preprint arXiv:2501.17887*, 2025.

[20] Linke Ouyang et al. OmniDocBench: Benchmarking diverse PDF document parsing with comprehensive annotations. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2412.07626.

[21] Lev Pevzner and Marti A. Hearst. A critique and improvement of an evaluation metric for text segmentation. *Computational Linguistics*, 28(1):19–36, 2002. doi: 10.1162/089120102317341756. URL `https://aclanthology.org/J02-1002/`.

[22] Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter W. J. Staar. DocLayNet: A large human-annotated dataset for document-layout segmentation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022. doi: 10.1145/3534678.3539043.

[23] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.

[24] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[25] Mateusz Śmigielski, Michał Rajkowski, Mateusz Zbrocki, Michał Bernacki-Janson, Karol Kunicki, Julianna Godziszewska, Maciej Piasecki, and Konrad Wojtasik. Chunking methods on retrieval-augmented generation — effectiveness evaluation against computational cost and limitations. *arXiv preprint arXiv:2606.00881*, 2026.

[26] Brandon Smith and Anton Troynikov. Evaluating chunking strategies for retrieval. Technical report, Chroma, 2024. URL `https://research.trychroma.com/evaluating-chunking`.

[27] Brandon Smock, Rohith Pesala, and Robin Abraham. PubTables-1M: Towards comprehensive table extraction from unstructured documents. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. arXiv:2110.00061.

[28] Junyuan Zhang and Others. OCR hinders RAG: Evaluating the cascading impact of OCR on retrieval-augmented generation. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. arXiv:2412.02592.

[29] Xu Zhong, Jianbin Tang, and Antonio Jimeno-Yepes. PubLayNet: Largest dataset ever for document layout analysis. In *International Conference on Document Analysis and Recognition (ICDAR)*, 2019. arXiv:1908.07836.